LINFO1104-MidTerm

Graded

Student
[redacted]

Total Points
19 / 21 pts

Question 1
Formal semantics
8.5 / 9 pts

1.1 Kernel language
2.5 / 3 pts

- – 0 pts Correct
- – 0.5 pts Small error
- – 0.5 pts Nesting error
- – 0.5 pts Local or declare instruction error
- ✔ – 0.5 pts Procedure argument(s) or syntax error in definition or call
- – 0.5 pts Constant value(s) error
- – 0.5 pts Browse call missing or wrong
- – 0.5 pts MakeAdd call missing or wrong
- – 1 pt Procedure definition missing or major errors in definition
- – 3 pts Incorrect

1.2 Abstract machine execution
3 / 3 pts

- ✔ – 0 pts Correct
- – 0.5 pts Small error
- – 1 pt Instruction wrong
- – 0.5 pts Browse instruction missing
- – 1 pt Environment wrong
- – 1 pt All or part of memory missing
- – 2 pts Procedure body missing (whole procedure executed)
- – 3 pts No resulting state given

1.3 Lambda calculus reduction
3 / 3 pts

- ✔ – 0 pts Correct
- – 1 pt Small errors
- – 2 pts Several errors but partly correct
- – 3 pts Too many errors
- – 3 pts Incorrect

Question 2
Definitions and program code
10.5 / 12 pts

2.1 Basic concepts
2 / 3 pts

- – 0 pts Correct
- – 1 pt Contextual environment definition incorrect
- ✔ – 1 pt List definition incorrect
- – 1 pt Scope definition incorrect
- – 3 pts Incorrect

2.2 FoldL function
2.5 / 3 pts

- – 0 pts Correct
- – 0.5 pts Small error(s) in definition
- – 1 pt Error(s) in definition
- – 1.5 pts Missing or wrong definition
- – 0.5 pts Wrong answer for tail-recursive.
- ✔ – 0.5 pts Wrong answer for contextual environment (FoldL is in the CE).
- – 0.5 pts Wrong answer for order (order is N+1 if F is order N).
- – 3 pts Incorrect

2.3 Invariant programming
6 / 6 pts

- ✔ – 0 pts Correct
- – 0.5 pts Small error(s) in program
- – 1 pt Error(s) in program
- – 2 pts Major errors in program
- – 3 pts Program missing
- – 1 pt Invariant mostly correct
- – 2 pts Invariant partly correct
- – 3 pts Invariant missing or wrong
- – 6 pts Incorrect

[redacted]

LINFO1104 Midterm
March 18, 2025

Q1. Formal semantics

Q1.1 Kernel language. [3pts]

Translate the following program:

```
local MakeAdd A in
   fun {MakeAdd A}
      fun {$ X} X+A end
   end
   A={MakeAdd 10}
   {Browse {A100}
end
end
```

into kernel language. Hint: your translation must use only kernel language instructions!

```
local MakeAdd A R R2 in
   MakeAdd = {proc {$ A R}
      R = {proc {$ X R2}
         R2 = X + A
      end}
   end}
   local Ten Hun R3 in
      Ten = 10
      {MakeAdd Ten A}
      Hun = 100
      {A Hun}
      {R3 A}
      {Browse R3}
   end
end
```

Q1.2 Abstract machine execution. [3pts]

Given the following execution state:

([({F X Y R},{F→*f*,X→x,Y→*y*,R→*r*}), ({Browse R},{Browse→*w*,R→*r*})],
{*f*=(**proc** {$ A B C} C=A+B+D **end**,{D→*d*}), *x=10, y=20, d=5, r, w*=(...definition of Browse...)})

Execute exactly **one step** in the abstract machine and give the resulting execution state. Hint: be careful to give the correct environment needed for computing the procedure.

```
([(C = A + B + D, {C → R, A → x, B → y, D → d}),
  ({Browse R}, {Browse → w, R → R})],
{f = (proc {$ A B C} C = A + B + D end, {D → d}),
 x = 10, y = 20, d = 5, 2, w = (... definition of Browse ...)})
```

[redacted]

Q1.3 Lambda calculus reduction. [3pts]
Given the following definitions:

n ≜ λf. λx. (f (f ... (f (f x))...)) *(f nested n times)*
TRUE ≜ λx. λy. x
FALSE ≜ λx. λy. y
PAIR ≜ λx. λy. λf. (f x y)
NIL ≜ λx. TRUE
SECOND ≜ λp. (p FALSE)

For this question, reduce the expression (SECOND (PAIR 2 NIL)). Show **all** the reduction steps. Use **abbreviations** as much as possible to make your answer as short and readable as possible (in particular, **only** replace an abbreviation by its expression when you need it for a reduction).

- (PAIR 2 NIL) ⇒ ((λx. λy. λf (f x y) 2) NIL) ⇒
  (λy. λf (f 2 y)) NIL) ⇒ λf. (f 2 NIL)
  PAIR 2 NIL reduces to λf. (f 2 NIL)

- (SECOND (PAIR 2 NIL) ⇒
  (λp. (p FALSE) (λf. (f 2 NIL)))
  ((λf. (f 2 NIL) FALSE)
  (FALSE 2 NIL)
  (((λx. λy. y) 2) NIL)
  NIL
  donc SECOND(PAIR 2 NIL) reduces to NIL

Q2. Definitions and program code

Q2.1 Basic concepts. [3pts]
Give precise definitions of the following concepts:

- Contextual environment
- List (EBNF grammar definition)
- Scope

- E_c → all free indentifiers that stock with procedure value
  for example, {inc = proc {$ X ?R} R = X+One end,
  {One → 0}
  ↳ E_c

- list - recursive structure that can be either nil or a „head" with „tail"
  <list> ::= nil | H"|"T
  for example 1|2|3|nil

- scope - the part of the program where the variable is defined locally, so it isn't visible anywhere except its scope
  for example local X in
  <s_1>
  <s_2>
  end

[redacted]

Q2.2 FoldL function. [3pts]
Define a function {FoldL L F U} that takes the list L=[a_0 a_1 a_2 a_3 ... a_{n-1}], the binary function F, and the value U, and returns {F ... {F {F U a_0} a_1} ... a_{n-1}}. Is your definition tail-recursive? What is the contextual environment of your definition? What is the order of your definition?

```
declare
fun {FoldL L F U}
   case L of nil then U
      [] H|T then {FoldL T F {F U H}}
   end
```

- yes the definition is tail-recursive, because it uses accumulator (U)
- $E_C = \{F \to f, L \to l, U \to u\}$
- the order of the function FoldL is 2

Q2.3 Invariant programming. [6pts]
Given a list L containing integers, define the function {Sum L A} that computes the sum of all elements of L using the accumulator A. Give the invariant of your function. Hint: the invariant divides the work between L and A. As L decreases, then A increases.

```
declare
fun {Sum L A}
   case L of nil then A
   [] H|T then {Sum T A+H}
   end
end
```

invariant $\{Sum\ L\ A\} = \sum_{i=j+1} a_i + A_j$

a_i → element of the list („i"th element}
A_j → accumulator to store sum of „j"th elements of the list